SYMBIOSIS INSTITUTE OF TECHNOLOGY, NAGPUR
Symbiosis International (Deemed University)
(Established under section 3 of the UGC Act, 1956)
Re-accredited by NAAC with 'A++' Grade | Awarded Category – I by UGC
Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

A PROJECT REPORT
ON

"Simple Weather Info Panel"

A project report submitted in partial fulfillment of the requirements for the degree of Bachelor of Technology in Computer Science and Engineering

BACHELOR OF TECHNOLOGY
COMPUTER SCIENCE AND ENGINEERING

Submitted By
Nimesh Maraskole[25070521068]
Divya Choudhary[25070521069]
Arnav Tuli[25070521070]

UNDER THE GUIDANCE OF
Prof. Piyush Chauhan

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
AY 2025-26

SYMBIOSIS INSTITUTE OF TECHNOLOGY, NAGPUR
Symbiosis International (Deemed University)
(Established under section 3 of the UGC Act, 1956)
Re-accredited by NAAC with 'A++' Grade | Awarded Category – I by UGC
Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the Project work entitled "Simple Weather Info Panel" is carried out by the Student Name, in partial fulfillment for the award of the degree of Bachelor of Technology in Computer Science and Engineering, Symbiosis International (Deemed University), Pune during the academic year 2025-2026.

Prof Piyush Chauhan

Dr. Gaurav Londhe
Associate Professor
Hack-O-Week In-Charge

Dr. Sagar Kumar Badhiye
HoD, Dept. of CSE, SIT Nagpur

Dr. Bhupesh Kumar Dewangan
Associate Professor, IPR Co-ordinator

DECLARATION

I hereby declare that the project titled "Simple Weather Info Panel" submitted to Symbiosis Institute of Technology, Constituent of Symbiosis International (Deemed University) Pune for the award of the degree of Bachelor of Technology in Computer Science and Engineering is a result of original research carried out by me. I understand that my report may be made electronically available to the public. It is further declared that the project report or any part thereof has not been previously submitted to any University or Institute for the award of any degree or diploma.

Name(s) of All Student(s) : Nimesh Marashkole, Divya Choudhary, Arnav Tuli

PRN : 25070521068, 25070521069, 25070521070

Degree: Bachelor of Technology in CSE

Department: Computer Science and Engineering

Title of the project : Simple Weather Info Panel

(Signature of the Student)

Date: 15-06-26

ABSTRACT

This project presents a Simple Weather Info Panel — a lightweight, static web application built using HTML, CSS, and . The panel displays simulated current weather information for three fixed cities: Mumbai, Delhi, and Pune. Since no external API or backend is involved, the application functions entirely on the client side, making it fast, offline-capable, and easily deployable.

The primary objective is to demonstrate how web technologies can be used to create visually appealing and informative dashboards without relying on complex infrastructure. The panel presents weather details such as temperature, humidity, weather condition, and wind speed in a clean card-based UI. Responsive design ensures compatibility across desktop and mobile devices.

This project serves as an educational prototype that can be extended in the future with real-time API integration, geolocation support, and a dynamic city selection feature.

Keywords: HTML5, CSS3, , Weather Dashboard, Static Web Application, Frontend Development, Responsive Design, No-API, Client-Side

TABLE OF CONTENTS

Section	Page
Certificate	i
Declaration	ii
Abstract	iii
Table of Contents	iv
Chapter 1: Background and Technical Overview	1
Chapter 2: Problem Statement and Motivation	3
Chapter 3: Novelty and Innovative Contributions	5
Chapter 4: Technical Advantages and Practical Usefulness	7
Chapter 5: Detailed Methodology / System Architecture	9
Chapter 6: Prior Art and Related Work (Literature Survey)	11
Chapter 7: Applications and Deployment Areas	13
Chapter 8: Conclusion and Future Scope	15
References (IEEE Format)	17
Appendices	18

CHAPTER 1: BACKGROUND AND TECHNICAL OVERVIEW

1.1 Introduction

The World Wide Web has become an indispensable platform for delivering information in real time. Among the many categories of information portals, weather dashboards rank among the most frequently accessed, offering users concise snapshots of atmospheric conditions. Traditional weather applications rely on external APIs and server-side logic to fetch live data. However, building a functional, visually rich weather panel without any backend is equally valuable — particularly in educational and resource-constrained environments.

This project, the Simple Weather Info Panel, is a fully client-side web application that presents weather data for three pre-defined Indian cities: Mumbai, Delhi, and Pune. All data is hardcoded in objects, eliminating any dependency on network requests, authentication tokens, or backend servers.

1.2 Core Technologies Used

- HTML5 – Structures the page layout, city cards, and data containers.
- CSS3 – Handles visual styling, gradient backgrounds, card shadows, and responsive grid layout.
- (ES6) – Stores weather data as JSON-like objects and dynamically injects values into the DOM.
- DOM Manipulation – Updates city card content at page load without any server round-trip.

1.3 Technical Overview

The application uses a three-card layout, where each card corresponds to a city. Each card renders city name, temperature (°C), weather condition (e.g., Sunny, Cloudy,

Rainy), humidity (%), and wind speed (km/h). CSS Flexbox or Grid is used to ensure cards are evenly spaced and stack vertically on smaller screens. loops through a data array and populates each card dynamically, making the codebase easy to extend.

CHAPTER 2: PROBLEM STATEMENT AND MOTIVATION

2.1 Problem Statement

In academic settings, students and teachers often require quick, accessible references to weather-related information for learning exercises, UI prototyping, and front-end development practice. Existing weather applications are either too complex, require API keys with rate limits, or demand internet connectivity. There is a clear need for a simple, self-contained weather dashboard that functions without any external dependency.

2.2 Motivation

The motivation behind this project stems from two key observations. First, many beginners learning web development struggle to find real-world projects that are simple enough to understand yet visually rewarding. A weather panel achieves this balance perfectly. Second, real-time weather APIs introduce complexities such as CORS handling, token management, and asynchronous programming that can distract learners from understanding the fundamentals of HTML, CSS, and interaction.

2.3 Objectives

- To design a weather dashboard that displays data for exactly three fixed cities.

- To implement the application using only HTML, CSS, and vanilla with no external libraries or APIs.
- To deliver a responsive, visually appealing UI accessible on all screen sizes.
- To serve as an educational template that can be extended to use real APIs in future iterations.

CHAPTER 3: NOVELTY AND INNOVATIVE CONTRIBUTIONS

3.1 No-API Architecture

The most distinctive feature of this project is its complete independence from any weather API. Unlike most weather applications that require OpenWeatherMap, WeatherAPI, or similar services, this panel stores all weather data as a object within the source code. This zero-dependency model means the application loads instantaneously, works offline, and requires no API keys — making it ideal for hackathons, demos, and classroom environments.

3.2 Data-Driven DOM Rendering

The project demonstrates a data-driven rendering approach where a single data structure (an array of city objects) drives the entire user interface. This pattern mirrors real-world frameworks like React or Vue, teaching students the concept of separating data from presentation — a cornerstone of modern frontend architecture.

3.3 Lightweight and Zero-Dependency

The entire application resides in a single HTML file (or three small files: index.html, style.css, script.js) with no npm packages, no build tools, and no bundlers. This

makes it trivially deployable on any web server, GitHub Pages, or even opened directly in a browser as a local file.

3.4 Scalable Data Structure

The hardcoded data is organized as a array of objects, each containing well-named keys. Adding a fourth or fifth city requires only adding another object to the array — no UI changes needed. This design demonstrates the open/closed principle at a beginner level.

CHAPTER 4: TECHNICAL ADVANTAGES AND PRACTICAL USEFULNESS

4.1 Technical Advantages

- Zero latency: No HTTP requests means data is available immediately on page load.
- Offline capability: The panel works without internet connectivity.
- Cross-platform: Runs on any device with a modern browser — desktop, tablet, or mobile.
- No CORS issues: Since no external fetch calls are made, cross-origin resource sharing errors are entirely avoided.
- Easy maintenance: Updating weather data requires modifying a single object.

4.2 Practical Usefulness

This panel is practically useful in several scenarios. In classrooms, it provides a concrete project for students learning HTML/CSS/JS. In hackathons, it serves as a rapid prototype that can be demonstrated without any setup. For UI/UX designers,

it acts as a mockup that realistically simulates a weather dashboard. In embedded or kiosk environments, it can be deployed without internet access.

4.3 Comparison with API-Based Weather Apps

Feature	This Project	API-Based App
Internet Required	No	Yes
API Key Needed	No	Yes
Load Speed	Instant	Depends on network
Real-Time Data	No (Simulated)	Yes
Complexity	Low	High

Table No.

CHAPTER 5: DETAILED METHODOLOGY / SYSTEM ARCHITECTURE

5.1 System Architecture

The application follows a purely client-side, single-tier architecture. There is no server, no database, and no API layer. The browser directly loads the HTML file, applies CSS styles, and executes to populate the weather cards.

5.2 Data Layer

Weather data is defined in script.js as a array of objects. Each object includes city name, temperature, condition (e.g., Sunny, Rainy, Cloudy), humidity percentage, and wind speed. For this project, the three cities are Mumbai, Delhi, and Pune, with representative seasonal data.

5.3 Presentation Layer

The HTML file contains a container div with three child card divs. Each card has placeholder elements (h2 for city name, spans for data values). CSS gives each card a gradient background, rounded corners, and a subtle box shadow. A weather condition icon (using emoji or icon fonts) provides visual context.

5.4 Logic Layer

On DOMContentLoaded, iterates over the data array and injects values into the corresponding card elements using document.getElementById or document.querySelectorAll. The mapping between data objects and card elements is achieved via data-city attributes or ordered array indexing.

5.5 Responsive Design

CSS media queries adjust the card layout from a three-column horizontal flex arrangement on desktops to a single-column stack on mobile screens (breakpoint at 768px). Font sizes, padding, and card widths scale accordingly.

5.6 File Structure

- index.html — Main HTML structure with card containers
- style.css — All visual styling including responsive media queries
- script.js — Weather data object and DOM injection logic

CHAPTER 6: PRIOR ART AND RELATED WORK (LITERATURE SURVEY)

6.1 Existing Weather Applications

Numerous weather applications exist in the commercial space. AccuWeather, Weather.com, and the Apple Weather app all rely on sophisticated backend services, real-time sensor data, and machine learning models for forecasting. While highly accurate, these applications are not suitable as learning references for beginner web developers due to their architectural complexity.

6.2 Educational Weather Projects

Several open-source educational projects on platforms like GitHub and CodePen demonstrate weather dashboards built with vanilla using the OpenWeatherMap API. These projects, while instructive, require students to manage API keys, handle asynchronous fetch calls, and deal with JSON parsing — barriers that can overwhelm beginners. The Simple Weather Info Panel removes these barriers entirely.

6.3 Related Tools and Technologies

Google's Firebase Hosting and GitHub Pages are popular platforms for deploying static HTML/CSS/JS projects. The project described here is directly compatible with both, requiring no build step. Similar no-backend dashboards have been created for cryptocurrency price trackers and sports scoreboards, demonstrating the viability of the pattern across domains.

6.4 Research on Frontend-Only Applications

Studies in web engineering have shown that frontend-only applications (JAMstack architecture) offer superior performance, security, and scalability for certain categories of applications [1][4]. While JAMstack typically leverages APIs at build

time, the underlying principle of serving pre-rendered HTML is analogous to this project's approach of embedding data directly in .

CHAPTER 7: APPLICATIONS AND DEPLOYMENT AREAS

7.1 Educational Use

The primary application is as a teaching aid in web development courses. The project demonstrates HTML structure, CSS layout, and DOM manipulation in a single, cohesive project. Instructors can assign it as a beginner project, and students can extend it incrementally — first adding more cities, then integrating a real API, and finally adding persistence with localStorage.

7.2 Hackathons and Prototypes

In hackathon environments where setup time is critical, this project can be submitted as a fully functional prototype within hours. The absence of any backend setup means participants can focus entirely on UI quality and feature differentiation.

7.3 Kiosk and Embedded Displays

The panel can be deployed on dedicated kiosk screens in offices, airports, or university lobbies where static weather data is updated periodically by administrators without requiring live API access.

7.4 Portfolio Projects

For students building their web development portfolios, a well-designed weather panel hosted on GitHub Pages serves as a visually impressive demonstration of

frontend skills. It is compact enough to be understood quickly by recruiters yet complex enough to showcase CSS layout and proficiency.

7.5 Offline or Low-Connectivity Environments

In rural or low-connectivity areas, offline-capable web apps provide access to critical information. A weather panel updated weekly by an administrator and served from a local network would deliver reliable weather guidance without internet dependency.

CHAPTER 8: CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

The Simple Weather Info Panel successfully demonstrates that a functional, visually appealing, and educationally valuable web application can be built without any backend, API, or external dependency. Using only HTML5, CSS3, and vanilla , the application presents weather information for three fixed Indian cities in a clean, responsive card-based layout.

The project achieves its primary goals: simplicity, accessibility, and extensibility. It serves as an excellent starting point for beginner web developers, a rapid hackathon prototype, and a foundation for more sophisticated weather applications. The separation of data from presentation in the layer reflects modern development best practices and prepares students for framework-based development.

8.2 Future Scope

- API Integration: Replace hardcoded data with live data from OpenWeatherMap or WeatherAPI using fetch() and async/await.

- Geolocation Support: Use the browser's Geolocation API to automatically detect the user's city and display relevant weather.
- Dynamic City Selection: Add a search bar or dropdown allowing users to select from a list of cities.
- 5-Day Forecast: Extend each city card to show a 5-day forecast with daily high/low temperatures.
- localStorage Persistence: Remember the user's preferred cities across browser sessions.
- Dark/Light Mode Toggle: Implement a theme switcher for improved accessibility and user preference.
- Progressive Web App (PWA): Add a service worker and manifest to make the panel installable and fully offline-capable.

REFERENCES (IEEE FORMAT)

[1] W3C, "HTML5 Specification," World Wide Web Consortium, 2014. [Online]. Available: https://www.w3.org/TR/html5/. [Accessed: Apr. 2026].

[2] W3C, "Cascading Style Sheets (CSS) – The Official Definition," World Wide Web Consortium, 2021. [Online]. Available: https://www.w3.org/Style/CSS/. [Accessed: Apr. 2026].

[3] ECMA International, "ECMAScript 2023 Language Specification," ECMA-262, 14th ed., ECMA International, Geneva, Switzerland, 2023. [Online]. Available: https://www.ecma-international.org/publications-and-standards/standards/ecma-262/. [Accessed: Apr. 2026].

[4] MDN Web Docs, " Guide," Mozilla Developer Network, 2024. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/ /Guide. [Accessed: Apr. 2026].

[5] MDN Web Docs, "CSS: Cascading Style Sheets," Mozilla Developer Network, 2024. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS. [Accessed: Apr. 2026].

APPENDICES:

Appendix A – Project File Structure

```
weather-panel/
│
├── index.html        → Main webpage with card
containers
├── style.css         → All styling and responsive
layout
├── script.js         → Weather data object & DOM
injection
└── README.txt        → Setup and deployment
instructions
```

Appendix B – System Requirements

Component	Requirement
Browser	Chrome / Firefox / Edge (latest)
Internet	Not required (fully offline)
Backend / Server	Not required

Operating System	Windows / macOS / Linux / Android / iOS

Appendix C: Glossary of Terms

Term	Definition
HTML	HyperText Markup Language — standard language for structuring web pages.
CSS	Cascading Style Sheets — defines visual presentation and layout of HTML elements.
	A scripting language that enables dynamic, interactive behaviour in web pages.
MCQ	Multiple Choice Question — a question with one correct answer among several options.
DOM	Document Object Model — a programming interface for HTML documents.